Appendix A
Character Strings in Python

A string is a sequence of characters that come from some ***alphabet***. In Python, the built-in **str** class represents strings based upon the Unicode international character set, a 16-bit character encoding that covers most written languages. Unicode is an extension of the 7-bit ASCII character set that includes the basic Latin alphabet, numerals, and common symbols. Strings are particularly important in most programming applications, as text is often used for input and output.

A basic introduction to the str class was provided in Section 1.2.3, including use of string literals, such as `'hello'`, and the syntax str(obj) that is used to construct a string representation of a typical object. Common operators that are supported by strings, such as the use of + for concatenation, were further discussed in Section 1.3. This appendix serves as a more detailed reference, describing convenient behaviors that strings support for the processing of text. To organize our overview of the str class behaviors, we group them into the following broad categories of functionality.

Searching for Substrings

The operator syntax, pattern **in** s, can be used to determine if the given pattern occurs as a substring of string s. Table A.1 describes several related methods that determine the number of such occurrences, and the index at which the leftmost or rightmost such occurrence begins. Each of the functions in this table accepts two optional parameters, start and end, which are indices that effectively restrict the search to the implicit slice s[start:end]. For example, the call s.find(pattern, 5) restricts the search to s[5:].

Calling Syntax	Description
s.count(pattern)	Return the number of non-overlapping occurrences of pattern
s.find(pattern)	Return the index starting the leftmost occurrence of pattern; else -1
s.index(pattern)	Similar to find, but raise ValueError if not found
s.rfind(pattern)	Return the index starting the rightmost occurrence of pattern; else -1
s.rindex(pattern)	Similar to rfind, but raise ValueError if not found

Table A.1: Methods that search for substrings.

Constructing Related Strings

Strings in Python are immutable, so none of their methods modify an existing string instance. However, many methods return a newly constructed string that is closely related to an existing one. Table A.2 provides a summary of such methods, including those that replace a given pattern with another, that vary the case of alphabetic characters, that produce a fixed-width string with desired justification, and that produce a copy of a string with extraneous characters stripped from either end.

Calling Syntax	Description
s.replace(old, new)	Return a copy of s with all occurrences of old replaced by new
s.capitalize( )	Return a copy of s with its first character having uppercase
s.upper( )	Return a copy of s with all alphabetic characters in uppercase
s.lower( )	Return a copy of s with all alphabetic characters in lowercase
s.center(width)	Return a copy of s, padded to width, centered among spaces
s.ljust(width)	Return a copy of s, padded to width with trailing spaces
s.rjust(width)	Return a copy of s, padded to width with leading spaces
s.zfill(width)	Return a copy of s, padded to width with leading zeros
s.strip( )	Return a copy of s, with leading and trailing whitespace removed
s.lstrip( )	Return a copy of s, with leading whitespace removed
s.rstrip( )	Return a copy of s, with trailing whitespace removed

Table A.2: String methods that produce related strings.

Several of these methods accept optional parameters not detailed in the table. For example, the replace method replaces all nonoverlapping occurrences of the old pattern by default, but an optional parameter can limit the number of replacements that are performed. The methods that center or justify a text use spaces as the default fill character when padding, but an alternate fill character can be specified as an optional parameter. Similarly, all variants of the strip methods remove leading and trailing whitespace by default, but an optional string parameter designates the choice of characters that should be removed from the ends.

Testing Boolean Conditions

Table A.3 includes methods that test for a Boolean property of a string, such as whether it begins or ends with a pattern, or whether its characters qualify as being alphabetic, numeric, whitespace, etc. For the standard ASCII character set, alphabetic characters are the uppercase A–Z, and lowercase a–z, numeric digits are 0–9, and whitespace includes the space character, tab character, newline, and carriage return. Conventions for what are considered alphabetic and numeric character codes are extended to more general Unicode character sets.

Calling Syntax	Description
s.startswith(pattern)	Return True if pattern is a prefix of string s
s.endswith(pattern)	Return True if pattern is a suffix of string s
s.isspace( )	Return True if all characters of nonempty string are whitespace
s.isalpha( )	Return True if all characters of nonempty string are alphabetic
s.islower( )	Return True if there are one or more alphabetic characters, all of which are lowercased
s.isupper( )	Return True if there are one or more alphabetic characters, all of which are uppercased
s.isdigit( )	Return True if all characters of nonempty string are in 0–9
s.isdecimal( )	Return True if all characters of nonempty string represent digits 0–9, including Unicode equivalents
s.isnumeric( )	Return True if all characters of nonempty string are numeric Unicode characters (e.g., 0–9, equivalents, fraction characters)
s.isalnum( )	Return True if all characters of nonempty string are either alphabetic or numeric (as per above definitions)

Table A.3: Methods that test Boolean properties of strings.

Splitting and Joining Strings

Table A.4 describes several important methods of Python's string class, used to compose a sequence of strings together using a delimiter to separate each pair, or to take an existing string and determine a decomposition of that string based upon existence of a given separating pattern.

Calling Syntax	Description
sep.join(strings)	Return the composition of the given sequence of strings, inserting sep as delimiter between each pair
s.splitlines( )	Return a list of substrings of s, as delimited by newlines
s.split(sep, count)	Return a list of substrings of s, as delimited by the first count occurrences of sep. If count is not specified, split on all occurrences. If sep is not specified, use whitespace as delimiter.
s.rsplit(sep, count)	Similar to split, but using the rightmost occurrences of sep
s.partition(sep)	Return (head, sep, tail) such that s = head + sep + tail, using leftmost occurrence of sep, if any; else return (s, '', '')
s.rpartition(sep)	Return (head, sep, tail) such that s = head + sep + tail, using rightmost occurrence of sep, if any; else return ('', '', s)

Table A.4: Methods for splitting and joining strings.

The join method is used to assemble a string from a series of pieces. An example of its usage is `' and '.join(['red', 'green', 'blue'])`, which produces the result `'red and green and blue'`. Note well that spaces were embedded in the separator string. In contrast, the command `'and'.join(['red', 'green', 'blue'])` produces the result `'redandgreenandblue'`.

The other methods discussed in Table A.4 serve a dual purpose to join, as they begin with a string and produce a sequence of substrings based upon a given delimiter. For example, the call `'red and green and blue'.split(' and ')` produces the result `['red', 'green', 'blue']`. If no delimiter (or None) is specified, split uses whitespace as a delimiter; thus, `'red and green and blue'.split()` produces `['red', 'and', 'green', 'and', 'blue']`.

String Formatting

The format method of the str class composes a string that includes one or more formatted arguments. The method is invoked with a syntax s.format(arg0, arg1, ...), where s serves as a ***formatting string*** that expresses the desired result with one or more placeholders in which the arguments will be substituted. As a simple example, the expression `'{} had a little {}'.format('Mary', 'lamb')` produces the result `'Mary had a little lamb'`. The pairs of curly braces in the formatting string are the placeholders for fields that will be substituted into the result. By default, the arguments sent to the function are substituted using positional order; hence, `'Mary'` was the first substitute and `'lamb'` the second. However, the substitution patterns may be explicitly numbered to alter the order, or to use a single argument in more than one location. For example, the expression `'{0}, {0}, {0} your {1}'.format('row', 'boat')` produces the result `'row, row, row your boat'`.

All substitution patterns allow use of annotations to pad an argument to a particular width, using a choice of fill character and justification mode. An example of such an annotation is `'{:-^20}'.format('hello')`. In this example, the hyphen (`-`) serves as a fill character, the caret (`^`) designates a desire for the string to be centered, and 20 is the desired width for the argument. This example results in the string `'-------hello--------'`. By default, space is used as a fill character and an implied $<$ character dictates left-justification; an explicit $>$ character would dictate right-justification.

There are additional formatting options for numeric types. A number will be padded with zeros rather than spaces if its width description is prefaced with a zero. For example, a date can be formatted in traditional "YYYY/MM/DD" form as `'{}/{:02}/{:02}'.format(year, month, day)`. Integers can be converted to binary, octal, or hexadecimal by respectively adding the character `b`, `o`, or `x` as a suffix to the annotation. The displayed precision of a floating-point number is specified with a decimal point and the subsequent number of desired digits. For example, the expression `'{:.3}'.format(2/3)` produces the string `'0.667'`, rounded to three digits after the decimal point. A programmer can explicitly designate use of fixed-point representation (e.g., `'0.667'`) by adding the character `f` as a suffix, or scientific notation (e.g., `'6.667e-01'`) by adding the character `e` as a suffix.

Appendix B
Useful Mathematical Facts

In this appendix we give several useful mathematical facts. We begin with some combinatorial definitions and facts.

Logarithms and Exponents

The logarithm function is defined as

$$\log_b a = c \qquad \text{if} \qquad a = b^c.$$

The following identities hold for logarithms and exponents:

1. $\log_b ac = \log_b a + \log_b c$
2. $\log_b a/c = \log_b a - \log_b c$
3. $\log_b a^c = c \log_b a$
4. $\log_b a = (\log_c a)/\log_c b$
5. $b^{\log_c a} = a^{\log_c b}$
6. $(b^a)^c = b^{ac}$
7. $b^a b^c = b^{a+c}$
8. $b^a/b^c = b^{a-c}$

In addition, we have the following:

Proposition B.1: *If $a > 0$, $b > 0$, and $c > a + b$, then*

$$\log a + \log b < 2\log c - 2.$$

Justification: It is enough to show that $ab < c^2/4$. We can write

$$\begin{aligned} ab &= \frac{a^2 + 2ab + b^2 - a^2 + 2ab - b^2}{4} \\ &= \frac{(a+b)^2 - (a-b)^2}{4} \leq \frac{(a+b)^2}{4} < \frac{c^2}{4}. \end{aligned}$$

■

The ***natural logarithm*** function $\ln x = \log_e x$, where $e = 2.71828\ldots$, is the value of the following progression:

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots.$$

In addition,

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$
$$\ln(1+x) = x - \frac{x^2}{2!} + \frac{x^3}{3!} - \frac{x^4}{4!} + \cdots .$$

There are a number of useful inequalities relating to these functions (which derive from these definitions).

Proposition B.2: *If* $x > -1$,

$$\frac{x}{1+x} \leq \ln(1+x) \leq x.$$

Proposition B.3: *For* $0 \leq x < 1$,

$$1 + x \leq e^x \leq \frac{1}{1-x}.$$

Proposition B.4: *For any two positive real numbers* x *and* n,

$$\left(1 + \frac{x}{n}\right)^n \leq e^x \leq \left(1 + \frac{x}{n}\right)^{n+x/2}.$$

Integer Functions and Relations

The "floor" and "ceiling" functions are defined respectively as follows:

1. $\lfloor x \rfloor$ = the largest integer less than or equal to x.
2. $\lceil x \rceil$ = the smallest integer greater than or equal to x.

The ***modulo*** operator is defined for integers $a \geq 0$ and $b > 0$ as

$$a \bmod b = a - \left\lfloor \frac{a}{b} \right\rfloor b.$$

The ***factorial*** function is defined as

$$n! = 1 \cdot 2 \cdot 3 \cdot \cdots \cdot (n-1)n.$$

The binomial coefficient is

$$\binom{n}{k} = \frac{n!}{k!(n-k)!},$$

which is equal to the number of different ***combinations*** one can define by choosing k different items from a collection of n items (where the order does not matter). The name "binomial coefficient" derives from the ***binomial expansion***:

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k}.$$

We also have the following relationships.

Proposition B.5: *If $0 \leq k \leq n$, then*

$$\left(\frac{n}{k}\right)^k \leq \binom{n}{k} \leq \frac{n^k}{k!}.$$

Proposition B.6 (Stirling's Approximation):

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1 + \frac{1}{12n} + \varepsilon(n)\right),$$

where $\varepsilon(n)$ is $O(1/n^2)$.

The ***Fibonacci progression*** is a numeric progression such that $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.

Proposition B.7: *If F_n is defined by the Fibonacci progression, then F_n is $\Theta(g^n)$, where $g = (1+\sqrt{5})/2$ is the so-called* **golden ratio**.

Summations

There are a number of useful facts about summations.

Proposition B.8: *Factoring summations:*

$$\sum_{i=1}^{n} af(i) = a\sum_{i=1}^{n} f(i),$$

provided a does not depend upon i.

Proposition B.9: *Reversing the order:*

$$\sum_{i=1}^{n}\sum_{j=1}^{m} f(i,j) = \sum_{j=1}^{m}\sum_{i=1}^{n} f(i,j).$$

One special form of is a ***telescoping sum***:

$$\sum_{i=1}^{n} (f(i) - f(i-1)) = f(n) - f(0),$$

which arises often in the amortized analysis of a data structure or algorithm.

The following are some other facts about summations that arise often in the analysis of data structures and algorithms.

Proposition B.10: $\sum_{i=1}^{n} i = n(n+1)/2$.

Proposition B.11: $\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6$.

Proposition B.12: *If $k \geq 1$ is an integer constant, then*

$$\sum_{i=1}^{n} i^k \text{ is } \Theta(n^{k+1}).$$

Another common summation is the ***geometric sum***, $\sum_{i=0}^{n} a^i$, for any fixed real number $0 < a \neq 1$.

Proposition B.13:

$$\sum_{i=0}^{n} a^i = \frac{a^{n+1}-1}{a-1},$$

for any real number $0 < a \neq 1$.

Proposition B.14:

$$\sum_{i=0}^{\infty} a^i = \frac{1}{1-a}$$

for any real number $0 < a < 1$.

There is also a combination of the two common forms, called the ***linear exponential*** summation, which has the following expansion:

Proposition B.15: *For $0 < a \neq 1$, and $n \geq 2$,*

$$\sum_{i=1}^{n} ia^i = \frac{a-(n+1)a^{(n+1)}+na^{(n+2)}}{(1-a)^2}.$$

The nth ***Harmonic number*** H_n is defined as

$$H_n = \sum_{i=1}^{n} \frac{1}{i}.$$

Proposition B.16: *If H_n is the nth harmonic number, then H_n is $\ln n + \Theta(1)$.*

Basic Probability

We review some basic facts from probability theory. The most basic is that any statement about a probability is defined upon a ***sample space*** S, which is defined as the set of all possible outcomes from some experiment. We leave the terms "outcomes" and "experiment" undefined in any formal sense.

Example B.17: *Consider an experiment that consists of the outcome from flipping a coin five times. This sample space has 2^5 different outcomes, one for each different ordering of possible flips that can occur.*

Sample spaces can also be infinite, as the following example illustrates.

Example B.18: *Consider an experiment that consists of flipping a coin until it comes up heads. This sample space is infinite, with each outcome being a sequence of i tails followed by a single flip that comes up heads, for $i = 1, 2, 3, \ldots$.*

A ***probability space*** is a sample space S together with a probability function Pr that maps subsets of S to real numbers in the interval $[0,1]$. It captures mathematically the notion of the probability of certain "events" occurring. Formally, each subset A of S is called an ***event***, and the probability function Pr is assumed to possess the following basic properties with respect to events defined from S:

1. $\Pr(\emptyset) = 0$.
2. $\Pr(S) = 1$.
3. $0 \leq \Pr(A) \leq 1$, for any $A \subseteq S$.
4. If $A, B \subseteq S$ and $A \cap B = \emptyset$, then $\Pr(A \cup B) = \Pr(A) + \Pr(B)$.

Two events A and B are ***independent*** if

$$\Pr(A \cap B) = \Pr(A) \cdot \Pr(B).$$

A collection of events $\{A_1, A_2, \ldots, A_n\}$ is ***mutually independent*** if

$$\Pr(A_{i_1} \cap A_{i_2} \cap \cdots \cap A_{i_k}) = \Pr(A_{i_1}) \Pr(A_{i_2}) \cdots \Pr(A_{i_k}).$$

for any subset $\{A_{i_1}, A_{i_2}, \ldots, A_{i_k}\}$.

The ***conditional probability*** that an event A occurs, given an event B, is denoted as $\Pr(A|B)$, and is defined as the ratio

$$\frac{\Pr(A \cap B)}{\Pr(B)},$$

assuming that $\Pr(B) > 0$.

An elegant way for dealing with events is in terms of ***random variables***. Intuitively, random variables are variables whose values depend upon the outcome of some experiment. Formally, a ***random variable*** is a function X that maps outcomes from some sample space S to real numbers. An ***indicator random variable*** is a random variable that maps outcomes to the set $\{0,1\}$. Often in data structure and algorithm analysis we use a random variable X to characterize the running time of a randomized algorithm. In this case, the sample space S is defined by all possible outcomes of the random sources used in the algorithm.

We are most interested in the typical, average, or "expected" value of such a random variable. The ***expected value*** of a random variable X is defined as

$$\mathbf{E}(X) = \sum_x x \Pr(X = x),$$

where the summation is defined over the range of X (which in this case is assumed to be discrete).

Proposition B.19 (The Linearity of Expectation): *Let X and Y be two random variables and let c be a number. Then*

$$E(X+Y) = E(X) + E(Y) \qquad \text{and} \qquad E(cX) = cE(X).$$

Example B.20: *Let X be a random variable that assigns the outcome of the roll of two fair dice to the sum of the number of dots showing. Then $\boldsymbol{E}(X) = 7$.*

Justification: *To justify this claim, let X_1 and X_2 be random variables corresponding to the number of dots on each die. Thus, $X_1 = X_2$ (i.e., they are two instances of the same function) and $\boldsymbol{E}(X) = \boldsymbol{E}(X_1 + X_2) = \boldsymbol{E}(X_1) + \boldsymbol{E}(X_2)$. Each outcome of the roll of a fair die occurs with probability $1/6$. Thus,*

$$\boldsymbol{E}(X_i) = \frac{1}{6} + \frac{2}{6} + \frac{3}{6} + \frac{4}{6} + \frac{5}{6} + \frac{6}{6} = \frac{7}{2},$$

for $i = 1, 2$. Therefore, $E(X) = 7$. ■

Two random variables X and Y are ***independent*** if

$$\Pr(X = x | Y = y) = \Pr(X = x),$$

for all real numbers x and y.

Proposition B.21: *If two random variables X and Y are independent, then*

$$\boldsymbol{E}(XY) = \boldsymbol{E}(X)\boldsymbol{E}(Y).$$

Example B.22: *Let X be a random variable that assigns the outcome of a roll of two fair dice to the product of the number of dots showing. Then $\boldsymbol{E}(X) = 49/4$.*

Justification: *Let X_1 and X_2 be random variables denoting the number of dots on each die. The variables X_1 and X_2 are clearly independent; hence*

$$\boldsymbol{E}(X) = \boldsymbol{E}(X_1 X_2) = \boldsymbol{E}(X_1)\boldsymbol{E}(X_2) = (7/2)^2 = 49/4.$$ ■

The following bound and corollaries that follow from it are known as ***Chernoff bounds***.

Proposition B.23: *Let X be the sum of a finite number of independent $0/1$ random variables and let $\mu > 0$ be the expected value of X. Then, for $\delta > 0$,*

$$\Pr(X > (1+\delta)\mu) < \left[\frac{e^{\delta}}{(1+\delta)^{(1+\delta)}} \right]^{\mu}.$$

Useful Mathematical Techniques

To compare the growth rates of different functions, it is sometimes helpful to apply the following rule.

Proposition B.24 (L'Hôpital's Rule): *If we have* $\lim_{n\to\infty} f(n) = +\infty$ *and we have* $\lim_{n\to\infty} g(n) = +\infty$, *then* $\lim_{n\to\infty} f(n)/g(n) = \lim_{n\to\infty} f'(n)/g'(n)$, *where* $f'(n)$ *and* $g'(n)$ *respectively denote the derivatives of* $f(n)$ *and* $g(n)$.

In deriving an upper or lower bound for a summation, it is often useful to ***split a summation*** as follows:

$$\sum_{i=1}^{n} f(i) = \sum_{i=1}^{j} f(i) + \sum_{i=j+1}^{n} f(i).$$

Another useful technique is to ***bound a sum by an integral***. If f is a nondecreasing function, then, assuming the following terms are defined,

$$\int_{a-1}^{b} f(x)\,dx \leq \sum_{i=a}^{b} f(i) \leq \int_{a}^{b+1} f(x)\,dx.$$

There is a general form of recurrence relation that arises in the analysis of divide-and-conquer algorithms:

$$T(n) = aT(n/b) + f(n),$$

for constants $a \geq 1$ and $b > 1$.

Proposition B.25: *Let* $T(n)$ *be defined as above. Then*

1. *If* $f(n)$ *is* $O(n^{\log_b a - \varepsilon})$, *for some constant* $\varepsilon > 0$, *then* $T(n)$ *is* $\Theta(n^{\log_b a})$.
2. *If* $f(n)$ *is* $\Theta(n^{\log_b a} \log^k n)$, *for a fixed nonnegative integer* $k \geq 0$, *then* $T(n)$ *is* $\Theta(n^{\log_b a} \log^{k+1} n)$.
3. *If* $f(n)$ *is* $\Omega(n^{\log_b a + \varepsilon})$, *for some constant* $\varepsilon > 0$, *and if* $af(n/b) \leq cf(n)$, *then* $T(n)$ *is* $\Theta(f(n))$.

This proposition is known as the ***master method*** for characterizing divide-and-conquer recurrence relations asymptotically.